INSTITUTO POLITÉCNICO DE SANTARÉM
Escola Superior de Gestão e Tecnologia

DOCUMENTO DE ARQUITETURA

Sistema de Gestão Hoteleira (SGH)

Deliverable 3



Autores:
Guilherme Caçador - 240000675
Rodrigo Silva - 240000762

Engenharia de Software | Docente: Pedro Martins
2025/2026

1. Visão Geral da Arquitetura

O Sistema de Gestão Hoteleira foi desenvolvido seguindo uma Arquitetura em Camadas (Layered Architecture), um estilo arquitetural clássico que promove a separação de responsabilidades e facilita a testabilidade, manutenibilidade e escalabilidade do sistema.

1.1. Justificação do Estilo Arquitetural

A escolha da arquitetura em camadas foi baseada nos seguintes fatores críticos:

- Complexidade do domínio: O sistema hoteleiro envolve múltiplos módulos (reservas, housekeeping, faturação, manutenção) com regras de negócio interrelacionadas que beneficiam de separação clara.
- Necessidade de testabilidade: A separação em camadas permite testar cada componente de forma isolada através de mocks e stubs, facilitando a manutenção da cobertura de testes acima de 70%.
- Requisitos de manutenibilidade: Alterações numa camada não afetam as restantes, desde que as interfaces (contratos) sejam respeitadas, reduzindo o risco de regressões.
- Compatibilidade com Spring Boot: A framework adotada segue naturalmente este padrão arquitetural com Controllers, Services e Repositories.

1.2. Camadas do Sistema

O sistema está organizado em cinco camadas distintas, cada uma com responsabilidades bem definidas:

1. Camada de Apresentação (Controllers)

Responsável pela interação com o utilizador através de endpoints REST API. Esta camada não contém lógica de negócio, apenas validação de entrada (DTOs), formatação de saída e mapeamento HTTP. Os controladores delegam toda a lógica para a camada de serviços.

2. Camada de Serviços (Services)

Contém toda a lógica de negócio do sistema. Implementa os casos de uso, aplica regras de validação e coordena as operações entre entidades. É aqui que residem os padrões Strategy (preços), Observer (notificações) e Factory (criação de objetos). Os serviços são transacionais e orquestram múltiplas operações de repositório.

3. Camada de Domínio (Entities)

Define as entidades principais do sistema (Quarto, Reserva, Hospede, Fatura, TicketManutencao) e as suas relações. Representa o modelo de domínio do negócio hoteleiro. As entidades são POJOs anotados com JPA para mapeamento objeto-relacional.

4. Camada de Persistência (Repositories)

Responsável pelo acesso à base de dados através de repositórios JPA. Abstrai os detalhes de persistência, permitindo trocar a implementação (ex: de H2 para PostgreSQL) sem afetar as camadas superiores. Utiliza Spring Data JPA para queries derivadas.

5. Camada de Infraestrutura

Gere integrações externas como serviços de email, gateways de pagamento, APIs de channel managers e configurações de segurança. Isola dependências externas do núcleo da aplicação.

1.3. Comparação de Estilos Arquiteturais

A tabela seguinte compara os estilos arquiteturais considerados durante o desenho do sistema:

Estilo	Características	Adequação ao SGH
Layered	Separação em camadas (apresentação, negócio, dados)	SELECIONADO - Ideal para aplicações empresariais com regras de negócio complexas
Microservices	Serviços independentes com APIs próprias	Excessivo para escala do projeto; overhead de deployment
MVC	Model-View-Controller para separação UI	Parcialmente aplicado na camada de apresentação REST
Hexagonal	Ports & Adapters para isolar domínio	Complexidade adicional sem benefício claro para o projeto

Tabela 1: Comparação de Estilos Arquiteturais

2. Padrões de Desenho (Design Patterns)

A robustez e extensibilidade do sistema são garantidas pela aplicação rigorosa de três padrões de desenho GoF (Gang of Four). Esta secção documenta cada padrão implementado, o problema que resolve, a implementação concreta e os benefícios obtidos.

2.1. Padrão Strategy (Comportamental)

Contexto: Cálculo de Preços Dinâmicos

Problema Identificado:

O hotel necessita de diferentes lógicas de cálculo de preço conforme a época do ano (alta, média, baixa), tipo de cliente (particular, empresarial, agência) e promoções especiais (Black Friday, early booking). Implementar esta lógica com múltiplos if/else ou switch statements tornaria o código difícil de manter, testar e estender, violando o princípio Open-Closed.

Solução Implementada:

Criámos a interface EstrategiaPreco que define o contrato para todos os algoritmos de cálculo. Cada estratégia concreta (EstrategiaPrecoNormal, EstrategiaPrecoSazonal, EstrategiaPrecoEmpresarial, EstrategiaPrecoPromocional) implementa a interface com a sua lógica específica. O ServicoPrecos recebe a estratégia apropriada em runtime através de injeção de dependências.

Implementação:

```
// Interface Strategy
public interface EstrategiaPreco {
    double calcular(double precoBase);
    String getNome();
    double getMultiplicador();
}

// Implementação Concreta - Época Alta
public class EstrategiaPrecoSazonal implements EstrategiaPreco {
    private static final double MULT_ALTA = 1.5;
    private static final double MULT_MEDIA = 1.2;
    private static final double MULT_BAIXA = 1.0;

    @Override
    public double calcular(double precoBase) {
        return precoBase * getMultiplicadorAtual();
    }

    private double getMultiplicadorAtual() {
        int mes = LocalDate.now().getMonthValue();
        if (mes >= 6 && mes <= 9) return MULT_ALTA;
```

```
        if (mes == 12 || mes <= 2) return MULT_MEDIA;
        return MULT_BAIXA;
    }
}
```

Localização no Código: Pacote com.sgh.estrategia

Benefícios Obtidos:

- Extensibilidade: Adicionar nova regra de preço (ex: DescontoNatal) requer apenas criar nova classe, sem modificar código existente
- Testabilidade: Cada estratégia pode ser testada isoladamente com valores conhecidos
- Manutenibilidade: Alterações numa estratégia não afetam as restantes

2.2. Padrão Observer (Comportamental)

Contexto: Sistema de Notificações

Problema Identificado:

Quando uma reserva é criada, cancelada, ou quando o estado de um quarto muda, múltiplos componentes do sistema precisam de ser notificados: o serviço de email deve enviar confirmação, o log de auditoria deve registar a operação, o painel da receção deve atualizar em tempo real, e o sistema de housekeeping deve ser alertado. Acoplar diretamente estes componentes violaria o princípio de Single Responsibility.

Solução Implementada:

O ServicoReserva e ServicoQuarto atuam como Subjects, mantendo uma lista de Observers registados. Quando o estado muda, todos os observers são notificados automaticamente através do método apropriado da interface. Cada observer decide o que fazer com a notificação.

Implementação:

```
// Interface Observer
public interface ObservadorReserva {
    void aoReservaCriada(Reserva reserva);
    void aoReservaCancelada(Reserva reserva);
    void aoCheckIn(Reserva reserva);
    void aoCheckOut(Reserva reserva);
}

// Observer Concreto - Notificações Email
public class ObservadorEmail implements ObservadorReserva {
    private final ServicoEmail servicoEmail;

    @Override
    public void aoReservaCriada(Reserva reserva) {
        String email = reserva.getHospede().getEmail();
```

```
            String assunto = "Confirmação Reserva #" + reserva.getId();
            servicoEmail.enviar(email, assunto, gerarCorpo(reserva));
        }
    }

    // Subject - Serviço de Reservas
    public class ServicoReserva {
        private List<ObservadorReserva> observadores = new ArrayList<>();

        public void adicionarObservador(ObservadorReserva obs) {
            observadores.add(obs);
        }

        private void notificarCriacao(Reserva reserva) {
            observadores.forEach(o -> o.aoReservaCriada(reserva));
        }
    }
```

Localização no Código: Pacote com.sgh.observador

Benefícios Obtidos:

- Desacoplamento: A lógica de negócio (reservas) está completamente separada da lógica de notificação
- Extensibilidade: Adicionar novo observer (ex: SMS, Slack) não requer alteração do subject
- Flexibilidade: Observers podem ser ativados/desativados dinamicamente em runtime

2.3. Padrão Factory Method (Criacional)

Contexto: Criação de Objetos Complexos

Problema Identificado:

A criação de diferentes tipos de quartos (Suite, Standard, Individual, Familiar, Presidencial) e faturas envolve lógica complexa de inicialização. Cada tipo de quarto tem preço base diferente, capacidade máxima, características específicas. Espalhar esta lógica pelo código duplicaria código e dificultaria a manutenção.

Solução Implementada:

As classes FabricaQuarto e FabricaFatura centralizam a instanciação dos objetos. O cliente solicita um objeto pelo tipo desejado, e a fábrica retorna a instância correta já configurada com todos os valores por omissão apropriados.

Implementação:

```
public class FabricaQuarto {
    private static long contadorId = 0;

    public static Quarto criarQuarto(TipoQuarto tipo, String numero, int
andar) {
        return switch (tipo) {
            case INDIVIDUAL -> new Quarto(gerarId(), numero, tipo, andar,
80.0, 1);
            case DUPLO -> new Quarto(gerarId(), numero, tipo, andar, 120.0,
2);
            case SUITE -> new Quarto(gerarId(), numero, tipo, andar, 200.0,
2);
            case FAMILIAR -> new Quarto(gerarId(), numero, tipo, andar,
180.0, 4);
            case PRESIDENCIAL -> new Quarto(gerarId(), numero, tipo, andar,
500.0, 2);
        };
    }

    private static long gerarId() {
        return ++contadorId;
    }
}

public class FabricaFatura {
    public static Fatura criarFatura(Reserva reserva, List<Consumo> extras)
{
        Fatura fatura = new Fatura();
        fatura.setReserva(reserva);
        fatura.setDataEmissao(LocalDateTime.now());
        fatura.adicionarLinha("Estadia", reserva.getValorTotal());
        extras.forEach(e -> fatura.adicionarLinha(e.getDescricao(),
e.getValor()));
```

```
        fatura.calcularTotais();
        return fatura;
    }
}
```

Localização no Código: Pacote com.sgh.fabrica

Benefícios Obtidos:

- Centralização: Toda a lógica de criação está num único local (Single Responsibility)
- Consistência: Objetos são sempre criados com estado válido e valores por omissão corretos
- Evolução: Adicionar novo tipo de quarto requer alteração apenas na fábrica

3. Aplicação dos Princípios SOLID

A arquitetura do SGH adere rigorosamente aos cinco princípios SOLID do desenho orientado a objetos, fundamentais para criar software manutenível, extensível e testável. Esta secção demonstra cada princípio com exemplos concretos da implementação.

3.1. S - Single Responsibility Principle (SRP)

Princípio: Uma classe deve ter apenas uma razão para mudar, ou seja, deve ter apenas uma responsabilidade.

Aplicação no SGH:

Separámos rigorosamente as responsabilidades em classes distintas. A classe Reserva é um objeto de dados que apenas guarda informação sobre a reserva. A classe ServicoReserva contém a lógica de negócio para criar, modificar e cancelar reservas. A classe PoliticaCancelamento calcula multas de cancelamento. A classe ServicoEmail envia notificações. Cada classe tem uma única responsabilidade bem definida.

Exemplo Concreto:

```
// Reserva - apenas dados
public class Reserva {
    private Long id;
    private Hospede hospede;
    private Quarto quarto;
    private LocalDate dataEntrada;
    private LocalDate dataSaida;
    private EstadoReserva estado;
    // getters e setters apenas
}

// ServicoReserva - lógica de negócio
public class ServicoReserva {
    public Reserva criar(DadosReserva dados) { ... }
    public void cancelar(Long id) { ... }
}

// PoliticaCancelamento - cálculo de multas
public class PoliticaCancelamento {
    public double calcularMulta(Reserva r, LocalDate dataCancelamento)
{ ... }
}
```

3.2. O - Open-Closed Principle (OCP)

Princípio: Entidades de software devem estar abertas para extensão, mas fechadas para modificação.

Aplicação no SGH:

O sistema de preços demonstra perfeitamente este princípio. O ServicoPrecos trabalha com a interface EstrategiaPreco, não com implementações concretas. Para adicionar uma nova regra de preço (ex: DescontoBlackFriday, DescontoFidelidade), basta criar uma nova classe que implementa a interface - zero alterações no código existente.

Exemplo Concreto:

```
// Código existente NÃO modificado
public class ServicoPrecos {
    public double calcular(Quarto q, EstrategiaPreco estrategia) {
        return estrategia.calcular(q.getPrecoBase());
    }
}

// Nova funcionalidade por EXTENSÃO
public class EstrategiaBlackFriday implements EstrategiaPreco {
    public double calcular(double precoBase) {
        return precoBase * 0.7; // 30% desconto
    }
}
```

3.3. L - Liskov Substitution Principle (LSP)

Princípio: Objetos de uma superclasse devem poder ser substituídos por objetos das suas subclasses sem afetar a correção do programa.

Aplicação no SGH:

Todas as implementações de EstrategiaPreco podem ser utilizadas de forma intercambiável. O ServicoPrecos não precisa de conhecer qual a implementação concreta - aceita qualquer objeto que implemente a interface. O comportamento é consistente independentemente da implementação.

Exemplo Concreto:

```
// Funciona com QUALQUER implementação de EstrategiaPreco
EstrategiaPreco estrategia1 = new EstrategiaPrecoNormal();
EstrategiaPreco estrategia2 = new EstrategiaPrecoSazonal();
EstrategiaPreco estrategia3 = new EstrategiaPrecoEmpresarial();

// Mesmo método, comportamento polimórfico
double preco1 = servicoPrecos.calcular(quarto, estrategia1);
double preco2 = servicoPrecos.calcular(quarto, estrategia2);
double preco3 = servicoPrecos.calcular(quarto, estrategia3);
```

3.4. I - Interface Segregation Principle (ISP)

Princípio: Clientes não devem ser forçados a depender de interfaces que não utilizam.

Aplicação no SGH:

Criámos interfaces específicas para cada contexto. A interface ObservadorReserva define apenas métodos relacionados com eventos de reserva. A interface ObservadorQuarto define apenas métodos para eventos de quartos. Assim, o ServicoLimpeza implementa apenas ObservadorQuarto, não sendo forçado a implementar métodos de reservas que não utiliza.

Exemplo Concreto:

```
// Interface segregada para reservas
public interface ObservadorReserva {
    void aoReservaCriada(Reserva r);
    void aoReservaCancelada(Reserva r);
}

// Interface segregada para quartos
public interface ObservadorQuarto {
    void aoEstadoAlterado(Quarto q, EstadoQuarto novoEstado);
}

// ServicoLimpeza só implementa o que precisa
public class ServicoLimpeza implements ObservadorQuarto {
    // Não é forçado a implementar métodos de reservas
}
```

3.5. D - Dependency Inversion Principle (DIP)

Princípio: Módulos de alto nível não devem depender de módulos de baixo nível. Ambos devem depender de abstrações.

Aplicação no SGH:

O ServicoReserva não depende diretamente de RepositorioReservaJPA (implementação concreta), mas sim da interface RepositorioReserva (abstração). Isto permite injetar diferentes implementações: RepositorioReservaJPA em produção, RepositorioReservaMemoria em testes. A inversão de controlo é gerida pelo Spring.

Exemplo Concreto:

```
// Abstração (interface)
public interface RepositorioReserva {
    Reserva guardar(Reserva r);
    Optional<Reserva> buscarPorId(Long id);
    List<Reserva> listarPorPeriodo(LocalDate inicio, LocalDate fim);
}

// Serviço depende da ABSTRAÇÃO, não da implementação
```

```
@Service
public class ServicoReserva {
    private final RepositorioReserva repositorio; // Interface!

    public ServicoReserva(RepositorioReserva repositorio) {
        this.repositorio = repositorio;
    }
}
```

4. Justificação da Pilha Tecnológica

A escolha das tecnologias foi baseada em critérios de maturidade, suporte da comunidade, adequação ao domínio e experiência da equipa de desenvolvimento.

Tecnologia	Versão	Justificação
Java	17 (LTS)	Linguagem fortemente tipada, ideal para sistemas empresariais. Versão LTS garante suporte de longo prazo.
Spring Boot	3.2	Framework com configuração simplificada, injeção de dependências nativa, ecossistema maduro (Security, Data, Web).
PostgreSQL	15	SGBD relacional robusto, gratuito, suporte completo a transações ACID, excelente desempenho.
Hibernate/JPA	6.4	ORM maduro, mapeamento objeto-relacional transparente, gestão automática de transações.
JUnit 5	5.10	Framework de testes padrão da indústria, suporte a testes parametrizados, integração com IDEs.
Mockito	5.8	Framework de mocking para testes unitários isolados, sintaxe fluente, verificação de interações.
JaCoCo	0.8.11	Análise de cobertura de código, relatórios HTML, integração com Maven e CI/CD.
Maven	3.9	Gestão declarativa de dependências, ciclo de vida de build padronizado, plugins ricos.
H2 Database	2.2	Base de dados em memória para testes de integração, compatível com PostgreSQL.

Tabela 2: Pilha Tecnológica do SGH

Alternativas Consideradas:

- Kotlin em vez de Java: Descartado por menor experiência da equipa, apesar de sintaxe mais concisa.
- MySQL em vez de PostgreSQL: PostgreSQL oferece melhor suporte a JSON e funcionalidades avançadas.
- Gradle em vez de Maven: Maven é mais familiar à equipa e tem maior adoção em projetos académicos.